

Graph of Thoughts: Improving LLM’s by Decoupling Logic from Memory

Jonas Lee
uhz7mq@virginia.edu
University of Virginia
Charlottesville, Virginia, USA

Neil Morgan
fwz8ya@virginia.edu
University of Virginia
Charlottesville, Virginia, USA

Abhinav Pappu
mzu9pt@virginia.edu
University of Virginia
Charlottesville, Virginia, USA

Paris Phan
auj4yx@virginia.edu
University of Virginia
Charlottesville, Virginia, USA

Andrew Thepvangs
htf6dp@virginia.edu
University of Virginia
Charlottesville, Virginia, USA

CCS Concepts: • **Computing methodologies** → *Natural language processing*; • **Theory of computation** → *Graph algorithms analysis*.

Keywords: large language models, graph algorithms, reasoning, supervised fine-tuning

1 Introduction and Project Refresher

Current Large Language Models (LLMs) fundamentally struggle with multi-step deterministic graph reasoning. When prompted to trace algorithms like BFS or Dijkstra’s, models frequently succumb to “state drift” [1], in which small errors within early steps compound to the point where the model begins hallucinating an entirely new graph.

To address this, the Graph of Thoughts (GoT) framework proposes decoupling the model’s logic capabilities from its memory constraints [2]. By utilizing a deterministic, non-neural state executor to manage the ground-truth graph state, the LLM is restricted strictly to predicting the optimal next algorithmic operation based on the induced local subgraph.

Our core deliverables are:

- A robust synthetic trace generation pipeline for graph algorithms.
- A model fine-tuned via Supervised Fine-Tuning (SFT) and negative sampling to interact seamlessly with a deterministic external state.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CS Capstone ’26, Charlottesville, VA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/02

<https://doi.org/XXXXXXX.XXXXXXX>

- A comprehensive benchmarking suite demonstrating GoT’s superiority over standard Chain of Thought (CoT) [3], primarily focusing on operational accuracy, state consistency, and structural generalization.

2 Progress to Date

2.1 Infrastructure and Dataset Generation

Our primary infrastructure and software architecture are fully initialized. We have implemented Python-based deterministic solvers for Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra’s algorithm. These solvers feature step-by-step state logging and graph-language token linearization, functioning as the foundational state executors for our training pipeline. A PyTorch-based SFT pipeline utilizing LoRA [4] for Llama 3.1B has also been successfully integrated on our GPU compute clusters.

Our most significant milestone is the complete generation of our large-scale JSON training datasets. We successfully generated deterministic algorithm traces across four primary network topologies: Erdős–Rényi graphs, Barabási–Albert (scale-free) graphs, Random Trees, and Grids. Furthermore, we implemented and generated adversarial traces including high-girth cycles, bottlenecks, and bridge graphs to ensure robust training.

We have integrated our evaluation frameworks with NL-Graph [5] and GLBench [6] to process these datasets under standard CoT paradigms, supplying us with benchmark failures to evaluate our GoT model against.

2.2 Generalization and Compute

An important open question concerns generalization across graph scales. Our current training examples are limited to graphs of size $n = 10\text{--}20$ nodes, and it remains to be seen whether models trained at this scale are capable of generalizing to substantially larger graphs. If generalization proves insufficient, we will expand our training examples to approximately $n = 200$ nodes and rerun the SFT pipeline accordingly. In terms of evaluation order, we plan to test using the Llama model first before proceeding to Qwen, as

Llama imposes a lower GPU memory footprint and allows for faster iteration. Should Llama-based testing fail to meet our quality thresholds, we will reevaluate our methods and training procedures before committing additional resources. Given the computational intensity of our full pipeline, these experiments require access to High-Performance Computing (HPC) infrastructure, to which we have recently acquired access, though we have not yet had the opportunity to begin runs.

3 Preliminary Results and Observations

Initial testing has focused on confirming our SFT pipeline and verifying that the model can learn the specialized graph-linearization syntax (e.g., $(u) - [w] \rightarrow (v)$). Our early training runs natively support Teacher Forcing on correct states, successfully preventing the compounding errors that normally plague long traces during the optimization phase.

Anecdotally, our tests on the CoT baseline datasets revealed that high-girth topologies cause immediate and catastrophic failure in standard LLM reasoning. Without external state management, the models quickly hallucinate short-cut edges that bridge long cycles. For example, when evaluating a DFS trace on a 20-node cycle:

- **CoT Output (Failed):** “Since we visited node 4, we observe neighbors [5, 19]. I will traverse to 19...” (Hallucinating a non-existent chordal edge).
- **GoT Output (Preliminary):** “Current Subgraph: (4)-[1]->(5). Target Operation: TRAVERSE(5).” (Enforced by state executor bounds).

4 Updated Timeline and Next Steps

Our immediate next steps heading toward the May 4 deadline include:

- **Model Scaling:** Transitioning our SFT configuration from Llama 3.1B to Qwen 2.5 7B Instruct [7].
- **Negative Sampling:** Activating our broken-trace injection pipeline to train the model to output the CORRECTION token upon detecting errors.
- **Comprehensive Evaluation:** Running zero-shot structural generalization assessments (training on Erdős-Rényi, testing on Barabási-Albert) and generating path-length vs. accuracy degradation plots using both GLBench and NLGraph.

Regarding schedule adjustments, data pipeline development and teacher-forcing integration required slightly more time than projected. Consequently, we are condensing our evaluation block (originally Weeks 6–8) by running our zero-shot transfer tests and faithfulness audits in parallel across multiple compute instances.

5 Conclusion and Requests for Feedback

Overall, the project remains very healthy and on track. The core software infrastructure is complete, the theoretical decoupling model is functional in early training phases, and we possess the requisite datasets to complete the full scope of our evaluation metrics.

At this juncture, we actively seek the committee’s feedback on our stress-testing suite. Specifically, we would appreciate insight on whether relying entirely on LoRA adapters for our zero-shot structural generalization tests might inappropriately bias the model’s capacity to extrapolate graph policies, compared to undertaking a more computationally expensive full-parameter fine-tuning.

References

- [1] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [2] Maciej Besta et al. Reasoning on graphs with large language models via graph-structured prompting. *arXiv preprint arXiv:2406.05673*, 2024.
- [3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [5] Yiming Wang, Zhuosheng Zhang, and Rui Wang. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems*, 2024.
- [6] Zhikai Chen, Haitao Mao, Jingzhe Liu, Yu Song, Bingheng Li, Wei Jin, Bahare Fatemi, Anton Tsitsulin, Bryan Perozzi, Hui Liu, and Jiliang Tang. GLBench: A comprehensive benchmark for graph with large language models. *arXiv preprint arXiv:2407.00869*, 2024.
- [7] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.